Module 22: Practice Problems

More About TypeScript

Problem 1

Problem statement: Declare a variable of type unknown holding a string. Use it to assert it as a string and get its length. Repeat using angle-bracket syntax <string>.

Input: let val: unknown = "Hello TypeScript";

Output: 16 (length), printed twice — once via as string, once via <string>.

Problem 2

Problem statement: Declare a string | number union variable. Cast it safely to number and add 10. Then write a double assertion (as unknown as) example and comment why it's risky.

Input: let value: string | number = "100";

Output: 110 (safe cast result); commented risky example, no runtime output required.

Problem 3

Problem statement: Create a Product interface with title: string, price: number, inStock?: boolean. Create two product objects — one with inStock, one without. Write a function that logs a product's price.

Input: { title: "Laptop", price: 55000 } and { title: "Mouse", price: 500, inStock: true }

Output: 55000 and 500 printed by the function.

Problem 4

Problem statement: Create a type alias PaymentMethod = "cash" | "card" | "mobile". Comment why interface can't express this. Create an Order interface using PaymentMethod.

Input: let method: PaymentMethod = "card";

Output: Order object like { id: 1, method: "card" }, type-checked successfully.

Problem 5

Problem statement: Write a generic function getLastElement<T> that returns the last element of an array. Test with number[] and string[].

Input: [10, 20, 30] and ["a", "b", "c"]

Output: 30 and "c"

Problem 6

Problem statement: Create a generic interface Container<T> with item: T. Create one Container<number> and one Container<string>.

Input: { item: 100 }, { item: "Books" }

Output: Container<number> = { item: 100 }, Container<string> = { item: "Books" }

Problem 7

Problem statement: Create HasId interface (id: number). Write findById<T extends HasId> that searches an array for a matching id. Write a call that violates the constraint and comment on the error.

Input: [{ id: 1, name: "A" }, { id: 2, name: "B" }], search id: 2

Output: { id: 2, name: "B" }; commented line shows constraint-violation error (e.g. passing an object without id).

Problem 8

Problem statement: Create a string enum OrderStatus: Placed, Shipped, Delivered, Cancelled. Write a function that prints a readable message for a given status.

Input: OrderStatus.Shipped

Output: "Current status: SHIPPED" (or equivalent readable message)

Problem 9

Problem statement: Create an appConfig object (theme, version), lock it with as const. Try mutating a property and comment on the resulting error. Derive a union type from an array using typeof + as const.

Input: const appConfig = { theme: "dark", version: 2 } as const;

Output: commented error on mutation attempt (Cannot assign to 'theme' because it is a read-only property); derived union type example (e.g. "red" | "green" | "blue").

Problem 10

Problem statement: Create Employee interface (name, id, salary, department). Write updateEmployee using Partial<Employee>. Build two new types with Pick<Employee, "name" | "id"> and Omit<Employee, "salary">.

Input: updateEmployee({ name: "Rafi" })

Output: logs { name: "Rafi" }; Pick type = { name, id }; Omit type = { name, id, department }
